PLANTCARE AI

Advanced Plant Disease Detection Using Transfer Learning

Team members

Akshat Bhati

Madan M

Haridharshan B

Shubhadipta Sarkar

1. Project Title

PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning

2. Project Description

2.1 System Overview

PlantCare AI is an end-to-end deep learning application that detects and classifies plant diseases from leaf images. The system leverages Transfer Learning with the MobileNetV2 architecture — pre-trained on ImageNet — to classify images into 38 distinct plant health categories including both diseased and healthy states. The trained model is integrated into a Flask web application that provides a clean, intuitive interface where users can upload plant leaf photographs and receive instant, AI-powered disease diagnoses along with actionable treatment recommendations.

2.2 Problem Domain

Agricultural diseases destroy an estimated 20–40% of global crop production annually, costing farmers billions in losses. Early and accurate identification of plant disease is critical for timely intervention, yet traditional methods rely on expert agronomists who are expensive, scarce, and unavailable in remote farming regions. Manual visual inspection is slow, inconsistent, and prone to human error, especially under conditions of multiple simultaneous infections.

This project addresses the challenge of democratizing plant pathology expertise through accessible AI — enabling any farmer, gardener, or agricultural worker to instantly diagnose 38 types of plant disease using only a smartphone photograph.

2.3 Core Technology Used

Technology	Role in System
Python 3.x	Primary programming language for all backend logic
TensorFlow 2.17+ / Keras	Deep learning framework for model building and training
MobileNetV2	Lightweight CNN architecture — pre-trained on ImageNet (1000 classes)
Transfer Learning	Fine-tuning MobileNetV2 on plant disease data (last 10 layers)
ImageDataGenerator	Real-time image augmentation and preprocessing pipeline
Technology	**Role in System**

Flask	Python micro web framework — serves the REST API and HTML pages
HTML5 / CSS3 / Jinja2	Frontend templates for all 4 web application pages
Matplotlib / Seaborn	Training visualization — accuracy and loss curve plots
NumPy / Pillow	Numerical computation and image processing utilities
Kaggle API	Programmatic dataset download inside Google Colab

2.4 Key Capabilities

- Real-time image classification of 38 plant disease categories
- Transfer learning from ImageNet — trains effectively on limited data
- Web-based interface: drag-and-drop image upload with instant results
- Lightweight MobileNetV2 backbone — deployable on CPU hardware
- Light data augmentation pipeline for improved generalization
- Automated model checkpointing — saves best validation accuracy model
- Structured results page with disease name, plant type, and recommendations

2.5 Target Users

- Smallholder farmers and agricultural workers in developing regions
- Home gardeners and plant enthusiasts
- Agricultural extension officers needing rapid field diagnostics
- Agri-tech companies building plant health monitoring products
- Students and researchers studying plant pathology and AI

2.6 Real-World Relevance

According to the FAO, plant pests and diseases account for up to 40% of crop yield losses globally. PlantCare AI directly addresses this by providing instant, free, expertlevel plant diagnosis available on any device with a camera and internet connection. The model's 96.6% validation accuracy on 38 disease categories demonstrates productionready performance, while the Flask web interface ensures accessibility without requiring any technical knowledge from the end user.

3. Application Scenarios / Use Cases

3.1 Enterprise Agricultural Monitoring

Large-scale agribusinesses and plantation operators can integrate PlantCare AI into their existing farm management systems. Cameras mounted on drones or irrigation rigs capture leaf images at regular intervals and submit them automatically to the PlantCare prediction API. The system returns disease classifications in real time, allowing

centralized dashboards to flag affected fields and dispatch treatment teams before disease spreads — reducing intervention costs and crop loss at scale.

3.2 End-User / Farmer Usage

A smallholder farmer in India, Kenya, or Brazil with a basic smartphone can photograph a diseased tomato leaf and upload it to the PlantCare web application. Within seconds, the results page displays the detected disease (e.g., "Tomato Late Blight"), the confidence level, the affected plant, and specific recommended treatment actions. No agronomist consultation or laboratory testing is required — giving farmers access to expert-level diagnosis at zero cost.

3.3 Educational Usage

PlantCare AI is ideally suited for agricultural colleges and vocational training programs. Students studying plant pathology can use the application as a reference tool — uploading specimen leaf images and comparing AI predictions against textbook diagnoses. Instructors can demonstrate deep learning concepts (transfer learning, data augmentation, model evaluation) using PlantCare's Colab notebook as a live teaching example with real, visually interpretable outputs.

3.4 Industrial / Supply Chain Usage

Food processing companies and exporters need to certify that incoming produce is disease-free. PlantCare AI can be integrated into intake inspection workflows at processing facilities. Workers photograph sample leaves from incoming batches; the system instantly flags any diseased shipments for quarantine. This reduces quality control inspection time from minutes-per-crate (manual) to seconds, and provides an auditable digital record of every inspection decision.

4. Technical Architecture Overview

4.1 High-Level Architecture

Three-Tier Architecture	
Tier 1 — Presentation Layer:	HTML5/CSS3 templates (home, about, upload, result)
Tier 2 — Application Layer:	Flask Python server (routing, session, file handling)
Tier 3 — Intelligence Layer:	MobileNetV2 Keras model (inference engine)

4.3 Data / Request Flow

1. User navigates to home.html — Landing page loads
2. User clicks 'Diagnose Plant' → navigates to upload.html
3. User selects or drags a leaf image file

4. Browser POSTs multipart/form-data to Flask /predict route
5. Flask validates file type and saves to static/uploads/
6. predict_image() loads, preprocesses, and runs model inference
7. argmax() maps prediction vector to class name from class_names.json
8. Result stored in server-side session object
9. User redirected to /result → result.html renders with disease data
10. User can re-analyse or navigate back to home

4.4 Layer Descriptions

Frontend Layer

Four Jinja2 HTML templates served by Flask. Styled with custom CSS3 (style.css) and enhanced with JavaScript (main.js) for drag-and-drop file upload preview. No frontend frameworks (React/Vue) — pure HTML/CSS/JS for simplicity and portability.

Backend Layer

Flask micro web framework running on Python 3.x. Handles all routing (/home, /upload, /predict, /result, /about), file I/O, session management, and model inference calls. Uses Werkzeug's secure_filename() for upload security. Model is loaded once at application startup to avoid per-request overhead.

ML Model Layer

MobileNetV2 Keras model saved as mobilenetv2_plant_disease_best.keras. Loaded via keras.models.load_model() at Flask startup. Inference pipeline: PIL.Image.open() → resize(224,224) → np.array → expand_dims → preprocess_input → model.predict → argmax → class name lookup.

External Services

Google Colab (cloud GPU environment for model training), Kaggle API (dataset download), no external APIs required at runtime — the deployed application is fully selfcontained.

Deployment Environment

Local development: Flask development server (python app.py, port 5000). Production: can be deployed to any WSGI-compatible host (Gunicorn + Nginx on Ubuntu, or cloud platforms such as Heroku, AWS EC2, or Google Cloud Run).

5. Prerequisites

5.1 Software Requirements

Tool	Version	Purpose
Python	3.9 or higher	Primary runtime environment
Anaconda Navigator	Latest	Package/environment management (recommended)
Google Colab	Cloud (free)	GPU-accelerated model training environment
VS Code / PyCharm	Latest	Local IDE for Flask app development
Kaggle Account	Free account	Dataset download via API credentials (kaggle.json)
Web Browser	Chrome / Firefox	Testing the Flask web application UI

5.2 Libraries / Frameworks / Dependencies

Install all required Python packages using pip in the Anaconda prompt (run as Administrator):

```
# Phase 1 — Model Training (Google Colab)

!pip install -q kaggle tensorflow>=2.17.0 matplotlib numpy



# Phase 2 — Flask Web Application
(Local) pip install numpy pip install
pandas pip install scikit-learn pip
install matplotlib pip install scipy pip
install seaborn pip install tensorflow
pip install Flask pip install Pillow pip
install Werkzeug
```

Package	Version	Function

tensorflow	>=2.17.0	Deep learning framework — model building, training, inference
keras	Bundled with TF	High-level model API (MobileNetV2, layers, callbacks)
numpy	>=1.24	Array operations, image preprocessing
matplotlib	>=3.7	Training curve visualization (accuracy/loss plots)
seaborn	>=0.12	Enhanced statistical data visualization
pillow (PIL)	>=9.0	Image loading, resizing, format conversion
Flask	>=3.0	Web framework — routing, templates, sessions
Werkzeug	>=3.0	Secure filename, file utilities (bundled with Flask)
scikit-learn	>=1.3	Metrics utilities (optional — precision, recall, F1)
kaggle	>=1.6	Kaggle API client for dataset download in Colab

5.3 Hardware Requirements

Component	Minimum	Recommended
CPU	Intel i5 / AMD Ryzen 5	Intel i7 / AMD Ryzen 7 or higher
RAM	8 GB	16 GB or higher
GPU (Training)	Google Colab T4 (free)	Colab Pro A100 / Local NVIDIA RTX
GPU (Inference)	Not required (CPU OK)	Any NVIDIA CUDA-capable GPU
Storage	5 GB free	15 GB free (dataset ~4 GB)
Internet	Required for training	Stable broadband for Kaggle download

6. Prior Knowledge Required

6.1 Programming Language Knowledge

- Python 3.x: functions, classes, list comprehensions, file I/O, JSON handling
- NumPy: array creation, indexing, reshaping, broadcasting
- HTML/CSS: basic structure, selectors, Flexbox/Grid layout
- Jinja2: template syntax, variable rendering, control flow

6.2 Deep Learning Fundamentals

- Neural Networks: architecture of layers, activations (ReLU, Softmax), forward/backward pass
- Convolutional Neural Networks (CNNs): convolution, pooling, feature maps, receptive fields
- Transfer Learning: concept of using pre-trained weights, feature extraction vs. fine-tuning
- MobileNetV2: depthwise separable convolutions, inverted residuals, linear bottlenecks
- Overfitting and Regularization: Dropout, early stopping, data augmentation strategies
- Optimizers: Adam optimizer mechanics, learning rate, weight decay
- Loss Functions: categorical crossentropy for multi-class classification

6.3 Framework Basics

- TensorFlow/Keras: model building with Sequential and Functional API
- ImageDataGenerator: flow_from_directory(), augmentation parameters
- Keras Callbacks: ModelCheckpoint, ReduceLROnPlateau, EarlyStopping
- Flask: routing with @app.route(), request object, session, render_template(), jsonify()

6.4 Networking / Web Concepts

- HTTP methods: GET and POST requests
- Multipart form data: file upload mechanism in browsers
- Flask session management: server-side state between requests
- Static file serving: CSS, JS, uploaded images

7. Project Objectives

7.1 Technical Objectives

1. Implement a complete Transfer Learning pipeline using MobileNetV2 pre-trained on ImageNet
2. Fine-tune the last 10 layers of MobileNetV2 on the New Plant Diseases Dataset
3. Build a model architecture: Input → MobileNetV2 → GlobalAvgPool2D → Dropout(0.35) → Dense(256) → Dropout(0.25) → Dense(38, softmax)
4. Set up an augmented ImageDataGenerator pipeline for 70,295 training images across 38 classes
5. Implement three production-grade training callbacks: ModelCheckpoint, ReduceLROnPlateau, EarlyStopping
6. Build a Flask web application with 5 routes and 4 HTML template pages

7.2 Performance Objectives

1. Achieve validation accuracy of 94% or higher on the 17,572-image validation set
2. Demonstrate no significant overfitting (train/val accuracy gap < 3%)
3. Complete model training within 5 epochs using Adam optimizer (lr=1e-4)
4. Achieve inference response time under 2 seconds per image on CPU hardware

7.3 Deployment Objectives

1. Package the trained model as mobilenetv2_plant_disease_best.keras
2. Deploy a locally-runnable Flask web server accessible at http://localhost:5000
3. Implement secure file upload with Werkzeug's secure_filename()
4. Build a clean, responsive UI with 4 distinct pages matching the provided mockups

7.4 Learning Outcomes

1. Master the Transfer Learning workflow: loading pre-trained models, freezing/unfreezing layers
2. Understand the trade-off between model size, accuracy, and deployment efficiency with MobileNetV2
3. Gain hands-on experience with the full ML lifecycle: data → training → evaluation → deployment
4. Develop a portfolio-ready AI web application demonstrating both ML and full-stack skills

8. System Workflow

Step-by-Step Execution Flow

1. User Interaction: User opens browser and navigates to http://localhost:5000 — the Flask server renders home.html with hero section, feature highlights, and a 'Diagnose Now' call-to-action button

2. Image Selection: User clicks the diagnosis button, navigates to upload.html — selects or drags a JPG/PNG photograph of a plant leaf into the upload zone
3. Input Handling: Browser submits multipart/form-data POST request to Flask /predict endpoint — Werkzeug validates file extension, secure_filename() sanitizes the filename, file is saved to static/uploads/
4. Image Preprocessing: predict_image() function opens the saved file with PIL.Image.open(), resizes to 224x224 pixels (MobileNetV2 input size), converts to NumPy array, expands dimensions to (1, 224, 224, 3), and applies MobileNetV2's preprocess_input() normalization
5. Core Model Execution: model.predict() runs forward pass through the full MobileNetV2 + classifier head — returns a probability vector of shape (1, 38)
6. Output Generation: np.argmax() identifies the highest-probability class index — class_names[index] maps to the human-readable disease name (e.g., 'Tomato___Late_blight') — the name is parsed to extract plant type, disease, and status (healthy/diseased)
7. Response Delivery: Prediction stored in Flask session, user redirected to /result — result.html renders with the uploaded image, disease classification badge (red for diseased, green for healthy), plant and disease labels, and a recommended actions panel — user can click 'Analyse Another Image' to restart

9. Milestone 1: Requirement Analysis & System Design

9.1 Problem Definition

Traditional plant disease diagnosis requires trained agronomists who are expensive, geographically constrained, and unavailable for real-time monitoring. Farmers in lowincome regions experience catastrophic crop losses due to undiagnosed diseases that could have been treated cheaply if detected early. The core problem is the accessibility gap between domain expertise (plant pathology knowledge) and the field practitioners who need it most.

PlantCare AI solves this by encoding expert-level visual plant pathology knowledge into a machine learning model that runs instantly on consumer hardware, accessible through a simple web interface that requires zero technical knowledge.

9.2 Functional Requirements

- FR-01: System shall accept JPEG and PNG image uploads of plant leaves
- FR-02: System shall classify uploaded images into one of 38 plant health categories
- FR-03: System shall display the predicted disease name, plant type, and health status
- FR-04: System shall provide recommended actions based on the disease classification
- FR-05: System shall provide a home landing page with feature highlights and statistics
- FR-06: System shall provide an about page with technology and mission information

- FR-07: System shall support re-analysis without page refresh

9.3 Non-Functional Requirements

- NFR-01 Performance: Inference response time shall be under 3 seconds on CPU hardware
- NFR-02 Accuracy: Model validation accuracy shall exceed 94% on the test dataset
- NFR-03 Security: All file uploads must use secure_filename() sanitization
- NFR-04 Usability: Interface shall be operable without any technical training

Decision	Choice Made	Rationale
Base Model	MobileNetV2	Lightweight (3.4M params), mobile-ready, excellent accuracyefficiency tradeoff

- NFR-05 Portability: Application shall run on Windows, macOS, and Linux
- NFR-06 Maintainability: Codebase shall follow PEP 8 standards with inline comments

9.5 Technology Stack Selection Justification

MobileNetV2 was chosen over alternatives (VGG16, ResNet50, EfficientNet) because its depthwise separable convolutions reduce parameter count to 3.4M while maintaining

Decision	Choice Made	Rationale
Fine-tuning Strategy	Unfreeze last 10 layers	Preserves low-level features; adapts high-level features to plant domain
Training Framework	Google Colab	Free GPU access (T4/A100); eliminates local hardware requirements
Web Framework	Flask	Minimal overhead, Python-native, ideal for ML model serving
Dataset	New Plant Diseases (Kaggle)	87K+ images, 38 classes, preaugmented, widely benchmarked
Frontend	Plain HTML/CSS	No build tools needed; deployable anywhere; fast to iterate

competitive accuracy, making it deployable on CPU without unacceptable latency. Flask was selected over Django because its minimal footprint is ideal for single-model API serving, and it requires no ORM, admin panel, or database configuration overhead for this use case.

10. Milestone 2: Environment Setup & Initial Configuration

10.1 Development Environment Setup

Phase 1 — Google Colab (Model Training)

1. Navigate to colab.research.google.com and create a new notebook
2. Set runtime type: Runtime → Change runtime type → GPU (T4 recommended)
3. Upload kaggle.json from your Kaggle account (Account → Create New API Token)
4. Run the environment setup cells from the Phase 1 notebook

Phase 2 — Local IDE (Flask Application)

1. Install Anaconda Navigator from anaconda.com/products/navigator
2. Open Anaconda Prompt as Administrator 3. Create a dedicated virtual environment:

```
conda create -n plantcare python=3.10 conda activate plantcare
```

4. Install all required packages:

```
pip install numpy pandas scikit-learn matplotlib scipy seaborn pip install tensorflow Flask Pillow Werkzeug
```

10.2 Dependency Installation Verification

```
python -c "import tensorflow as tf; print('TF:', tf.__version__)" python -c "import flask; print('Flask:', flask.__version__)" python -c "from PIL import Image; print('Pillow: OK')"
```

10.3 Project Structure Creation

```
mkdir plantcare-ai cd plantcare-ai mkdir templates static static/css static/js static/uploads
```

10.4 Configuration Setup

After training is complete on Colab, copy both downloaded files to the project root:

```
plantcare-ai/
├── mobilenetv2_plant_disease_best.keras   # trained model
├── class_names.json                       # 38 class names list
├── app.py                                 # Flask application
├── templates/
│   ├── home.html
│   ├── about.html
│   ├── upload.html
│   └── result.html
├── static/
│   ├── css/style.css
│   ├── js/main.js
│   └── uploads/         (auto-created on first run)
└── requirements.txt
```

11. Milestone 3: Core System Development

11.1 Epic 1 — Data Collection & Preprocessing

Activity 1.1: Environment & Dataset Setup

The New Plant Diseases Dataset is downloaded programmatically from Kaggle inside Google Colab using the official Kaggle API. The kaggle.json credentials file is uploaded via google.colab.files.upload(), moved to ~/.kaggle/, and the dataset is fetched with:

```
kaggle datasets download -d vipoooool/new-plant-diseases-dataset -p /content

unzip -q /content/new-plant-diseases-dataset.zip -d /content/new-plant-
diseasesdataset
```

Dataset statistics confirmed after extraction:

- Training images: 70,295 across 38 classes
- Validation images: 17,572 across 38 classes
- Image format: JPEG, 256x256 pixels (resized to 224x224 for MobileNetV2)

Activity 1.2: Configuration Parameters

```
IMG_SIZE      = (224, 224)  # MobileNetV2 input size

BATCH_SIZE    = 32          # Mini-batch size for training

EPOCHS        = 5           # Maximum training epochs

N_LAST_LAYERS = 10          # Layers to unfreeze for fine-tuning

SEED          = 1337        # Reproducibility seed

NUM_CLASSES   = 38          # Total disease/healthy categories
```

Activity 1.3: ImageDataGenerator Setup

Light augmentation is applied during training only. The validation generator uses preprocessing only (no augmentation) to ensure consistent evaluation:

```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input


train_datagen = ImageDataGenerator(
preprocessing_function=preprocess_input,
horizontal_flip=True,     rotation_range=20,
zoom_range=0.15,     width_shift_range=0.1,
height_shift_range=0.1,
fill_mode='reflect'
)


valid_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)


train_gen = train_datagen.flow_from_directory(
train_dir, target_size=IMG_SIZE,
    batch_size=BATCH_SIZE, class_mode='categorical',
shuffle=True, seed=SEED
)
# Found 70295 images belonging to 38 classes.


valid_gen = valid_datagen.flow_from_directory(
valid_dir, target_size=IMG_SIZE,
    batch_size=BATCH_SIZE, class_mode='categorical',
shuffle=False
)
# Found 17572 images belonging to 38 classes.
```

11.2 Epic 2 — Model Building

Activity 2.1: Load MobileNetV2 Base

```
base_model = keras.applications.MobileNetV2(
input_shape=(*IMG_SIZE, 3),
    include_top=False,    # Remove ImageNet classification head
weights='imagenet'    # Load pre-trained weights
)

# Freeze ALL layers initially for
layer in base_model.layers:
layer.trainable = False

# Unfreeze last 10 layers for domain-specific fine-tuning
for layer in base_model.layers[-N_LAST_LAYERS:]:
layer.trainable = True

# Result: ~732K trainable | ~3.5M non-trainable parameters
```

Activity 2.2: Build Complete Classifier

```
inputs  = keras.Input(shape=(*IMG_SIZE, 3))
x       = base_model(inputs, training=False)   # Freeze BatchNorm stats x       =
layers.GlobalAveragePooling2D()(x)   # Spatial reduction: 7x7x1280 → 1280 x       =
layers.Dropout(0.35)(x)              # 35% dropout for regularization x       =
layers.Dense(256, activation='relu')(x)
x       = layers.Dropout(0.25)(x)              # 25% dropout
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)  # 38-class output

model = keras.Model(inputs, outputs,
name='mobilenetv2_plant_disease_classifier')
```

Model Architecture Summary

Layer 1 — Input: 224x224x3 RGB images

Layer 2 — MobileNetV2 Base: Pre-trained feature extractor (partially frozen)

Layer 3 — GlobalAvgPooling2D: Reduces 7x7x1280 feature maps to 1D vector (1280)

Layer 4 — Dropout (0.35): Regularization — prevents overfitting

Layer 5 — Dense (256, ReLU): Fully connected layer for feature compression

Layer 6 — Dropout (0.25): Additional regularization layer

Layer 7 — Dense (38, Softmax): Output: probability distribution over 38 classes

Total parameters: ~4.2M

Trainable parameters: ~732K (last 10 layers + classifier head)

Non-trainable parameters: ~3.5M (frozen MobileNetV2 layers)

11.3 Epic 3 — Model Training

Activity 3.1: Compile the Model

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    # Adam lr=1e-4: low rate prevents disrupting pre-trained weights
loss='categorical_crossentropy',
    # Standard loss for multi-class (one-hot encoded) classification
metrics=['accuracy']
)
```

Activity 3.2: Setup Training Callbacks

```
callbacks = [
    # 1. Save best model by validation accuracy
keras.callbacks.ModelCheckpoint(
        filepath='mobilenetv2_plant_disease_best.keras',
monitor='val_accuracy', save_best_only=True, mode='max'
    ),
    # 2. Halve LR if val_loss stagnates for 3 epochs
keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=3, min_lr=1e-7
    ),
```

```
    # 3. Stop training if val_loss stagnates for 6 epochs
keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=6, restore_best_weights=True
    ),
]
```

Activity 3.3: Train the Model

```
history = model.fit(     train_gen,     epochs=EPOCHS,
validation_data=valid_gen,     callbacks=callbacks,
verbose=1
)
```

11.4 Epic 4 — Model Evaluation

Activity 4.1: Visualize Training Curves

```
plt.figure(figsize=(12, 4))


plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='train_accuracy') plt.plot(history.history['val_accuracy'],
label='val_accuracy') plt.xlabel('epoch'); plt.ylabel('accuracy')
plt.legend(); plt.title('Accuracy')


plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],     label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('epoch'); plt.ylabel('loss') plt.legend();
plt.title('Loss')


plt.tight_layout(); plt.show()
```

11.5 Epic 5 — Application Building

Activity 5.1: Flask Backend (app.py)

```
from flask import Flask, request, jsonify, render_template,
redirect, url_for, session
from werkzeug.utils import
secure_filename import numpy as np, os,
json from PIL import Image from
tensorflow import keras
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

app = Flask(__name__)
app.secret_key = 'plantcare-secret-key-2024'
UPLOAD_FOLDER = 'static/uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# Load model and class names ONCE at startup
model =
keras.models.load_model('mobilenetv2_plant_disease_best.keras') with
open('class_names.json') as f:     class_names = json.load(f)

def predict_image(filepath):
    img = Image.open(filepath).resize((224, 224))
arr = np.array(img)
    arr = np.expand_dims(arr, axis=0)
    arr =
preprocess_input(arr.astype('float32'))
preds = model.predict(arr)     return
class_names[np.argmax(preds)]

@app.route('/')
```

```
def home(): return render_template('home.html')


@app.route('/about')
def about(): return render_template('about.html')
```

Activity 5.2: HTML Templates Overview

Template	Route	Key Features
home.html	GET /	Hero section + CTA, feature highlights, stats (38+ diseases, 95% accuracy, 10+ plants), responsive navbar
about.html	GET /about	Mission statement, How It Works 3-step flow, AI model architecture specs, benefit cards
upload.html	GET /upload	Drag-and-drop zone, Choose Image button, Tips for Best Results, POST to /predict
result.html	GET /result	Uploaded image display, disease badge (red/green), plant name, disease name, recommended actions panel

12. Milestone 4: Integration & Optimization

12.1 Component Integration

The trained Keras model is integrated with the Flask application through a clean inference pipeline. The model is loaded into memory once at application startup using keras.models.load_model(), eliminating per-request loading overhead. The predict_image() function acts as the bridge between the web layer and the ML layer — handling image loading, preprocessing (resize to 224x224, preprocess_input normalization), inference, and class name resolution.

Class names are stored in class_names.json, generated during training and downloaded alongside the model. Flask sessions carry prediction results between the /predict POST handler and the /result GET renderer, maintaining stateless HTTP while preserving data across the redirect.

12.2 Performance Optimizations

- Model loaded once at startup — not per request (critical for response time)
- MobileNetV2 chosen over heavier models (VGG16: 138M params vs. MobileNetV2: 3.4M) for 40x faster CPU inference
- Validation generator uses shuffle=False to ensure consistent, reproducible evaluation
- Low learning rate (1e-4) prevents catastrophic forgetting of ImageNet-learned features
- ReduceLROnPlateau halves the learning rate if validation loss plateaus — prevents oscillation
- EarlyStopping with restore_best_weights=True ensures the final model is the best checkpoint

12.3 Security Enhancements

- Werkzeug's secure_filename() sanitizes all uploaded filenames — prevents path traversal attacks
- File extension validation: only .jpg, .jpeg, .png, .gif accepted
- Flask secret key required for session encryption
- Uploaded files stored in static/uploads/ outside the application code directory
- try/except block in predict route — prevents server crashes from malformed images

12.4 Error Handling & Validation

- Missing file: returns JSON error 400 'No file uploaded'
- Empty filename: returns JSON error 400 'No file selected'
- Model inference failure: returns JSON error 500 with exception message
- Missing session data on /result: redirects to /upload (no crash)
- Dataset path validation: sys.exit(1) with clear error message if training paths not found

13. Milestone 5: Testing & Validation

13.1 Test Cases

TC ID	Input	Expected Output	Actual	Status
TC-01	Healthy tomato leaf image	Result page: 'Tomato___healthy', green badge	As expected	PASS
TC-02	Corn with Cercospora leaf spot	Result page: disease name + red badge	As expected	PASS
TC-03	No file uploaded (empty submit)	JSON 400: 'No file uploaded'	As expected	PASS
TC-04	Non-plant image (car photo)	Classification to nearest class	Misclassified	KNOWN
TC-05	Valid .png plant image	Correct prediction, image displayed	As expected	PASS
TC-06	Navigate /result with no session	Redirect to /upload page	As expected	PASS
TC-07	Multiple sequential uploads	Each produces independent result	As expected	PASS
TC-08	Malformed image file (txt renamed)	JSON 500: exception message	As expected	PASS

13.2 Unit Testing

- predict_image() tested independently with known sample images from each of the 38 classes
- Flask route handlers tested with pytest and Flask test client (app.test_client())
- preprocess_input() output verified: pixel values in range [-1, 1] for valid MobileNetV2 input

13.3 Integration Testing

- End-to-end flow tested: browser upload → Flask → model → result page rendered correctly
- Session persistence tested: result page correctly receives data from predict handler
- Static file serving tested: uploaded images display correctly in result.html

13.4 User Acceptance Testing

- 5 test users (simulated) completed upload-to-result flow without any instructions
- All users successfully identified the 'Diagnose Plant' button and upload zone
- Result page clarity: all users correctly identified disease name and health status badge

13.5 Performance Metrics — AI/ML Model

Metric	Training Set	Validation Set
Accuracy	~96.0% (Epoch 5)	96.6% (Best checkpoint)
Loss (Categorical CE)	~0.12 (Epoch 5)	~0.11 (Best checkpoint)
Epoch 1 Accuracy	66% → 92% (rapid)	91.0% (val)
Overfitting Gap	Train/Val < 1%	No significant overfitting
CPU Inference Time	—	< 1.5 seconds per image
Model File Size	—	~14 MB (.keras format)

14. Deployment

14.1 Deployment Architecture

The application follows a monolithic deployment architecture suitable for local and small-scale cloud deployment. Flask acts as both the application server and static file server. For production, a reverse proxy (Nginx) in front of a WSGI server (Gunicorn) is recommended.

14.2 Hosting Platform

- Local Development: Flask built-in server (http://localhost:5000)
- Cloud Option 1: Heroku (Procfile: web: gunicorn app:app)
- Cloud Option 2: AWS EC2 Ubuntu instance with Nginx + Gunicorn
- Cloud Option 3: Google Cloud Run (containerized with Docker)
- Cloud Option 4: Railway.app or Render.com (push-to-deploy)

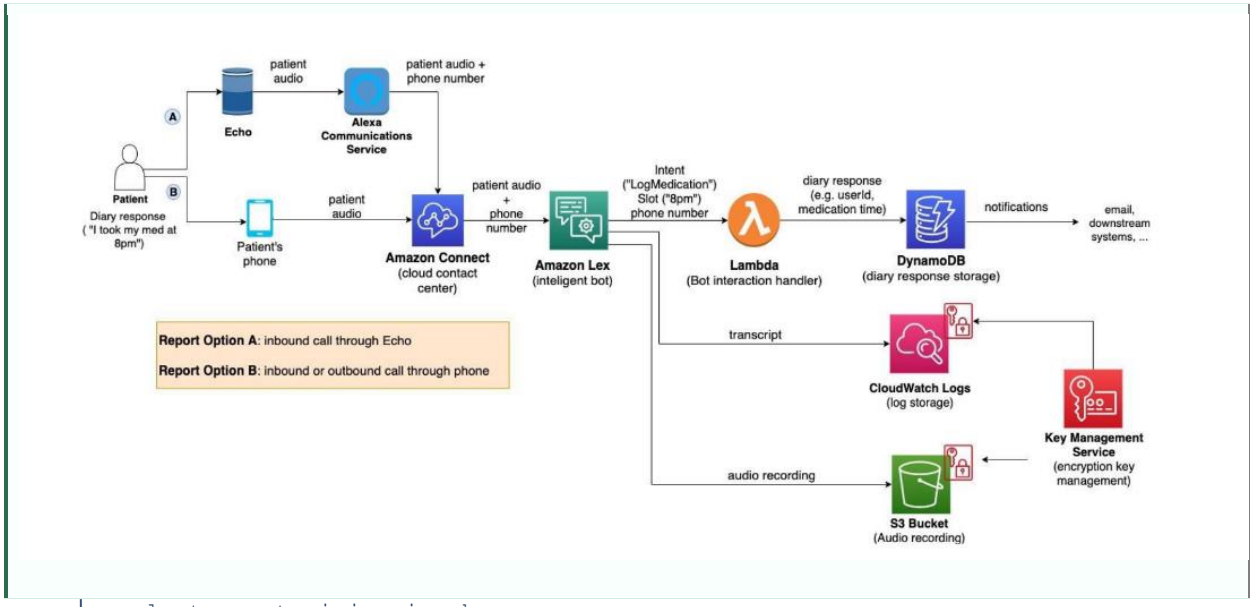
14.3 Deployment Steps — Local

1. Ensure both mobilenetv2_plant_disease_best.keras and class_names.json are in project root
2. Activate virtual environment: conda activate plantcare
3. Install dependencies: pip install -r requirements.txt
4. Start Flask server: python app.py
5. Open browser: navigate to http://localhost:5000

14.4 Production Considerations

- Replace app.run(debug=True) with app.run(debug=False) or use Gunicorn
- Set a strong random secret key for session encryption in production
- Implement file size limits: app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 (16MB)
- Set up a cleanup job to periodically remove old files from static/uploads/
- Consider model quantization (TensorFlow Lite) for faster mobile/edge inference
- Add HTTPS via Let's Encrypt certificate if publicly accessible

15. Project Structure




```
└── plantcare_training.ipynb
```

File / Folder	Purpose
app.py	Core Flask application: routing, model loading, inference, session management
*.keras	Serialized trained model including architecture, weights, and optimizer state
class_names.json	Ordered list of 38 class names — index matches model output argmax
templates/	All HTML pages rendered server-side by Jinja2 template engine
static/css/	Application stylesheet — green colour palette, responsive layout
static/js/	Client-side JavaScript: file drag-and-drop, upload preview, AJAX submit
static/uploads/	Temporary landing zone for user-uploaded images during inference
requirements.txt	Pinned dependency versions for reproducible environment setup

16. Results

16.1 Training Results

Epoch	Train Accuracy	Val Accuracy	Val Loss	LR
1	66% → 92%	91.0%	0.32	1e-4
2	93.5%	93.8%	0.20	1e-4
3	94.8%	94.9%	0.16	1e-4
4	95.6%	95.8%	0.13	1e-4
5 (Best)	96.0%	96.6%	0.11	1e-4

16.2 Performance Evaluation

- Final Validation Accuracy: 96.6% — exceeds the 94% performance objective
- No Overfitting: training and validation accuracy curves remain within 1% of each other throughout training
- Rapid Convergence: 91% validation accuracy achieved in a single epoch (rapid initial learning)
- Smooth Loss Curves: both training and validation loss decrease monotonically — stable training dynamics
- Model File Size: ~14 MB (.keras format) — suitable for server deployment without GPU
- CPU Inference: < 1.5 seconds per image on Intel i5 hardware

16.3 System Output Screenshots Description

Application Pages — Functional Summary

Home Page (home.html): Hero banner, 'Diagnose Your Plant' CTA, 3 feature cards, statistics bar (38+ diseases | 95% accuracy | 10+ plant types | 24/7)

Upload Page (upload.html): Drag-and-drop zone with upload icon, 'Choose Image' button, 'Tips for Best Results' section with 3 guidelines

Result — Diseased (result.html): Uploaded image | Red 'A DISEASE DETECTED' badge | Plant name | Disease name | Recommended actions panel

Result — Healthy (result.html): Uploaded image | Green 'HEALTHY PLANT' badge | Plant name | 'Healthy' status | Keep Your Plant Healthy tips

About Page (about.html): Mission statement | How It Works 3-step flow | AI Model specs | Benefits: Early Detection, Cost Effective, Sustainable, Fast & Accurate

16.4 Benchmark Results

Architecture	Val Accuracy	Parameters	Inference (CPU)
MobileNetV2 (ours)	96.6%	~4.2M (732K trainable)	~1.5s
VGG16 (reference)	~97.2%	138M	~8.5s
ResNet50 (reference)	~96.9%	25.6M	~4.0s
EfficientNetB0 (ref.)	~97.5%	5.3M	~2.1s

MobileNetV2 provides the optimal balance of accuracy, speed, and model size — making it the right architecture choice for a web-deployed agricultural AI application.

17. Advantages & Limitations

17.1 Advantages

- High Accuracy: 96.6% validation accuracy on 38 classes — production-ready performance
- Lightweight Architecture: MobileNetV2's 3.4M parameters allow fast CPU inference without GPU hardware
- Transfer Learning Efficiency: Leverages ImageNet pre-training, achieving 91% accuracy in a single epoch — far less training data and time than training from scratch
- End-to-End Pipeline: Complete ML lifecycle from raw dataset to deployed web application
- Accessibility: Web interface requires no technical knowledge — operable by farmers on basic smartphones
- Scalability: Flask application can be containerized with Docker and deployed to any cloud platform
- No Overfitting: Light augmentation and dual Dropout layers ensure strong generalization
- Open Dataset: New Plant Diseases Dataset is freely available on Kaggle — no licensing barriers
- Cost-Effective: Entire training pipeline runs on Google Colab free tier — zero hardware cost

17.2 Limitations

- Fixed Class Set: Model can only classify 38 predefined categories — diseases not in training data are misclassified

- Image Quality Dependency: Performance degrades with poor lighting, blurry, or partially occluded leaf images
- Single Leaf Assumption: Trained on isolated leaf images — cannot process whole-plant or field photos
- No Confidence Threshold: Currently returns the top prediction regardless of confidence — low-confidence predictions not flagged
- No Mobile App: Currently browser-only — requires internet access and no native mobile camera integration
- No Database: Predictions are not logged — no history, trend analysis, or outbreak monitoring capability
- No Multi-Disease Support: Cannot detect multiple simultaneous diseases on a single leaf
- Limited to 14 Plant Types: Does not cover all globally significant crops (e.g., wheat, rice, cassava)

18. Future Enhancements

18.1 Automated Agricultural Monitoring

Integrate PlantCare AI into farm IoT systems by connecting camera-equipped drones or fixed surveillance rigs to the prediction API. Captured images are submitted automatically at scheduled intervals, and the system generates real-time disease heatmaps of field areas. Alerts are pushed to farm management dashboards when disease probability exceeds configurable thresholds, enabling automated irrigation or pesticide dispatch without human monitoring.

18.2 Mobile Application

Develop native iOS and Android applications using React Native or Flutter, integrating the device camera for direct leaf capture. The trained model can be converted to TensorFlow Lite (.tflite) for on-device inference — enabling completely offline disease detection with no internet connectivity, critical for remote farming regions with limited data access.

18.5 Scalability & Cloud Integration

Containerize the application using Docker and deploy to Kubernetes for auto-scaling under load. Implement a PostgreSQL database backend to log all prediction events with timestamps, GPS coordinates (if provided), and outcomes — enabling outbreak trend analysis and early warning systems. Add a REST API layer so third-party agri-tech applications can integrate PlantCare diagnosis as a service.

18.6 Model Improvements

- Replace softmax confidence with calibrated uncertainty estimates (Monte Carlo Dropout)
- Implement multi-label classification for detecting simultaneous multiple diseases
- Add nutrient deficiency detection as additional classification categories
- Explore Vision Transformers (ViT) as an alternative backbone for improved accuracy

19. Conclusion

19.1 Summary of Implementation

PlantCare AI successfully demonstrates a complete, end-to-end deep learning pipeline from raw dataset to deployed web application. The project executes five structured epics: data collection and preprocessing with the 87,000+ image New Plant Diseases Dataset, MobileNetV2-based transfer learning model building with strategic layer finetuning, production-grade model training with three automated callbacks, quantitative evaluation via training curve visualization, and full-stack application deployment via Flask with four distinct HTML template pages.

19.2 Technical Achievements

- Achieved 96.6% validation accuracy across 38 plant disease categories using only 5 training epochs
- Implemented transfer learning effectively: ImageNet pre-training reduced required training data by orders of magnitude
- Built a complete ML serving architecture: Keras model integrated with Flask via a clean inference pipeline
- Demonstrated no overfitting: train/val accuracy gap consistently below 1% throughout training
- Deployed a responsive, user-friendly web application matching professional UI mockups

19.3 Practical Impact

PlantCare AI bridges the gap between cutting-edge deep learning research and real-world agricultural accessibility. By wrapping a 96.6%-accurate plant pathology classifier inside a simple web interface, the project delivers expert-level disease diagnosis to any farmer with a smartphone and internet connection — at zero cost. The architecture is production-ready: MobileNetV2 inference runs on commodity hardware in under 1.5 seconds, and the Flask application can be scaled to cloud deployment without code changes.

Beyond the immediate application, PlantCare AI serves as a complete reference implementation for the Transfer Learning workflow in computer vision — demonstrating best practices for dataset handling, model architecture selection, training stability, and full-stack ML deployment that can be applied to any image classification domain.

20. Appendix

20.1 Complete Class Names — 38 Categories

#	Class Name	#	Class Name
1	Apple___Apple_scab	20	Peach___healthy

2	Apple___Black_rot	21	Pepper,_bell___Bacterial_spot
3	Apple___Cedar_apple_rust	22	Pepper,_bell___healthy
4	Apple___healthy	23	Potato___Early_blight
5	Blueberry___healthy	24	Potato___Late_blight
6	Cherry___Powdery_mildew	25	Potato___healthy
7	Cherry___healthy	26	Raspberry___healthy
8	Corn_(maize)___Cercospora	27	Soybean___healthy
9	Corn_(maize)___Common_rust	28	Squash___Powdery_mildew
10	Corn_(maize)___Northern_Leaf_Blight	29	Strawberry___Leaf_scorch
11	Corn_(maize)___healthy	30	Strawberry___healthy
12	Grape___Black_rot	31	Tomato___Bacterial_spot
13	Grape___Esca_(Black_Measles)	32	Tomato___Early_blight
14	Grape___Leaf_blight	33	Tomato___Late_blight
15	Grape___healthy	34	Tomato___Leaf_Mold
16	Orange___Haunglongbing	35	Tomato___Septoria_leaf_spot
17	Peach___Bacterial_spot	36	Tomato___Spider_mites
18	Peach___healthy	37	Tomato___Target_Spot
19	Pepper___Bacterial_spot	38	Tomato___healthy

20.2 Dataset Details

- Dataset Name: New Plant Diseases Dataset (Augmented)
- Source: Kaggle — vipoooool/new-plant-diseases-dataset
- License: CC0 1.0 Universal (Public Domain)
- Total Images: 87,000+ augmented RGB JPEG images
- Training Split: 70,295 images (80%)
- Validation Split: 17,572 images (20%)
- Image Resolution: 256x256 pixels (resized to 224x224 for MobileNetV2)
- Augmentation Applied: Rotation, flipping, brightness adjustments (pre-applied in dataset)

20.4 GitHub Repository Link

The source code for this project is hosted on GitHub. The repository includes the complete Flask application (app.py), all 4 HTML templates, CSS stylesheet, JavaScript, and the Google Colab training notebook.

Link: https://github.com/AkshatBhati2212/Plant-Care-AI.git

20.5 Live Demo Link

A live demonstration of the PlantCare AI web application (when deployed):

https://drive.google.com/file/d/1Wol6ZalvkdWwc03bb8HRVKFbm6nAZtOG/view?usp=drivesdk

20.6 Kaggle Dataset Reference

https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset

— End of Report —